arXiv:2511.17048v1 [cs.CV] 21 Nov 2025

RoomPlanner: Explicit Layout Planner for Easier LLM-Driven 3D Room Generation

Wenzhuo Sun[1*], **Mingjian Liang**[1*], **Wenxuan Song**[2], **Xuelian Cheng**[1†], **Zongyuan Ge**[1]

[1]Monash University, Melbourne, Australia
[2]The Hong Kong University of Science and Technology(GZ), Guangzhou, China

Abstract

In this paper, we propose *RoomPlanner*, the first fully automatic 3D room generation framework for painlessly creating realistic indoor scenes with only short text as input. Without any manual layout design or panoramic image guidance, our framework can generate explicit layout criteria for rational spatial placement. We begin by introducing a hierarchical structure of language-driven agent planners that can automatically parse short and ambiguous prompts into detailed scene descriptions. These descriptions include raw spatial and semantic attributes for each object and the background, which are then used to initialize 3D point clouds. To position objects within bounded environments, we implement two arrangement constraints that iteratively optimize spatial arrangements, ensuring a collision-free and accessible layout solution. In the final rendering stage, we propose a novel AnyReach Sampling strategy for camera trajectory, along with the Interval Timestep Flow Sampling (ITFS) strategy, to efficiently optimize the coarse 3D Gaussian scene representation. These approaches help reduce the total generation time to under 30 minutes. Extensive experiments demonstrate that our method can produce geometrically rational 3D indoor scenes, surpassing prior approaches in both rendering speed and visual quality while preserving editability. The code will be available soon.

Introduction

Text-to-3D scene generation aims to convert textual descriptions into 3D environments. Designed to lower costs and reduce the required expertise, this paradigm is conducive to various downstream applications, such as embodied AI (Chen et al. 2024; Wang et al. 2024), gaming (Yu et al. 2025), filmmaking, and augmented / virtual reality (Nießner et al. 2013). Benefitting from the remarkable success of generation models (Ho, Jain, and Abbeel 2020; Song, Meng, and Ermon 2020) and implicit neural representation (Mildenhall et al. 2021; Kerbl et al. 2023), numerous methods have emerged to address the challenges of text-to-3D object generation, achieving significant milestones. However, directly transitioning from generating 3D assets to creating complete scenes is not a trivial task. This is partly due to the fact that layout information requires additional context and human effort. Based on the extracted layout sources, existing 3D scene generation methods can be broadly classified into two categories: **I.** *Visual-guided*, and **II.** *Rule-based* methods.

Visual-guided methods rely on extracting implicit spatial information from visual representations, *e.g.,* multi-view RGB or panorama images. Methods in this category typically adopt a two-stage generation framework, where image generation serves as an intermediate process, followed by an implicit neural representation network that renders a 3D scene from previously generated images. Some methods (Li et al. 2024b; Huang et al. 2025) treat objects and backgrounds as an indivisible whole as input for the implicit rendering network. As a result, these continuous representations produced by these methods lack object-level decoupling, which significantly restricts practical use. These kinds of approaches can also lead to foggy or distorted geometry, as shown in Figure 1 (b). Instead of being directly constrained, the layout configuration is learned from a text-to-image generation model, which is optimized by aligning the generated images with the input text using a similarity score. Thus, the optimization of the scene configuration is constrained by the capabilities of the image generation model, making it difficult to directly control the layout through textual descriptions.

Rule-based methods, on the other hand, explicitly utilize predefined rules based on physical relationships and constraints to create layout information of 3D environments. Early methods (Li et al. 2024a; Liu et al. 2023; Cohen-Bar et al. 2023; Höllein et al. 2023) depend on manually crafted prompt templates or human-designed layouts for initialization, which helps generate physically plausible scenes. This procedure relies on specialized expert knowledge and typically involves a tedious trial-and-error process to refine predefined spatial attributes of objects, *e.g.,* position, size, and rotation *etc.*. This human-in-the-loop paradigm increases the demand for human labor while also undermining the end-to-end training process.

On a separate line of research and to reduce human effort in layout design, data-driven approaches treat scene synthesis as a 3D asset arrangement task within a bounded environment. These methods add objects sequentially using an order prior with a forest structure (Sun et al. 2024) or simultaneously through conditional score evaluation (Maillard et al. 2024). Recently, Large Language Models (LLMs) have been

*These authors contributed equally. † Corresponding author: Xuelian.Cheng@monash.edu

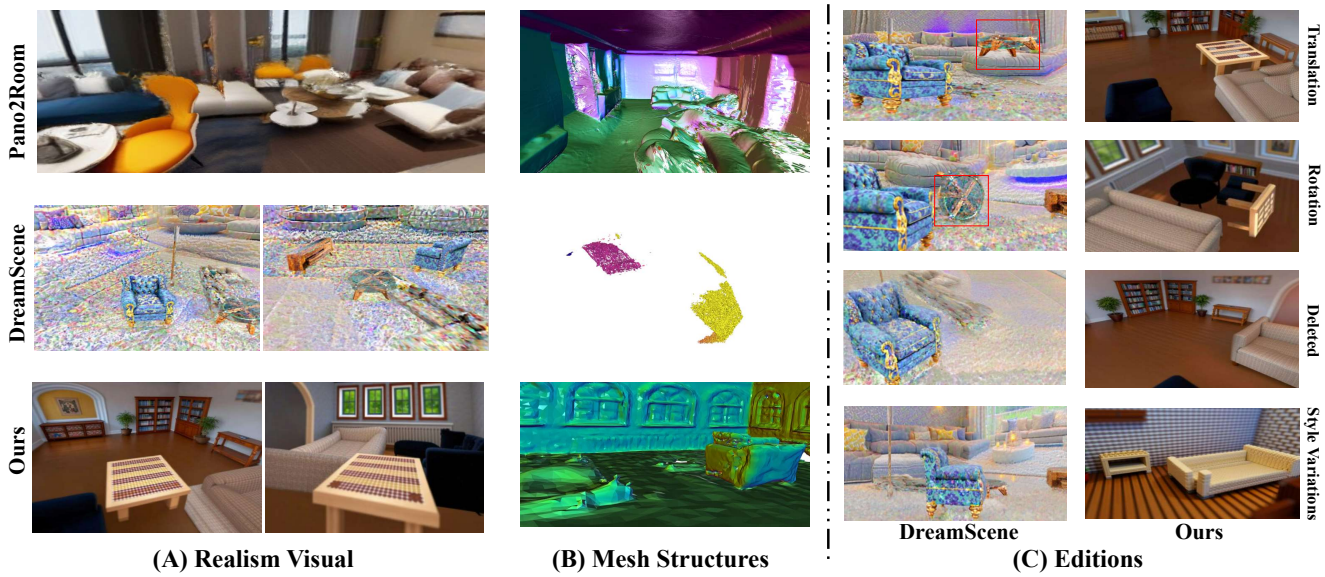


Figure 1: Compared to previous methods, *ie.,*visual-guided method Pano2Room (Pu, Zhao, and Lian 2024) and rule-based method DreamScene (Li et al. 2024a), our approach effectively generates indoor scenes characterized by (a) more realism, (b) smoother mesh structures, and (c) support for a diverse array of editions, including operations such as rotation, translation, importing/deleting 3D assets, and style variations.

introduced to automate the creation of 3D scenes (Yang et al. 2024c,b; Sun 2025). LLMs can directly translate unstructured language into structured spatial representations, which reduces dependence on rigid templates. However, their reliance on existing 3D assets limits the versatility and flexibility required to generate diverse and innovative scenes.

In this paper, we present **RoomPlanner**, which combines the advances of visual-guided and rule-based models. Unlike previous approaches, our method enables automatic generation of differentiable 3D scenes and layout constraints given only short textual prompts. Our framework adopts a 'Reasoning-Grounding, Arrangement, and Optimization' pipeline to break down the complex 3D scene generation task into controllable and scalable subtasks. In the reasoning and grounding stage, our hierarchical LLM-driven agent planners can parse short user prompts into executable long-term descriptions. Specifically, our high-level LLM agent produces textual descriptions that select the initial target scene and objects that satisfy the contextual semantics. The low-level LLM agent generates fine-grained grounding descriptions that infer object and background scales, along with layout rules that match real-world dimensions. These explicit physical constraints also ensure that the generated 3D scenes meet the interaction requirements.

After obtaining the initial scene description from the LLM agents, we utilize a text-to-3D object generator (Jun and Nichol 2023) to create 3D assets represented as point clouds. To assign initial object assets into the scene, we then employ interactive optimization to update the layout design by verifying whether the generated layout meets our constraints. To improve the rationality of the indoor scene, we first apply collision constraints to detect overlaps between objects. Additionally, we introduce a reachability constraint to assess path viability between entry points and target assets, supporting dynamic asset placement through closed-loop optimization. The generated scenes remain fully editable at both the object and global levels, allowing for reposition, insertion, deletion, and style transfer, as shown in Figure 1 (c).

For photorealistic rendering, we introduce a module that integrates Interval Timestep Flow Sampling (ITFS) with reflection flow matching and hierarchical multi-timestep sampling. By leveraging a 2D diffusion prior and detailed scene descriptions from LLM agents, this module generates continuous 3D representations that exhibit a realistic appearance and rich semantics. We also propose the AnyReach camera trajectory sampling strategy that can optimize both object-centric and global views within a single rendering pass. Consequently, our pipeline produces high-fidelity scenes in a single optimization stage, reducing the overall runtime by nearly 2× compared to prior work.

Related Work

Text-to-3D Object Generation

Existing 3D scene generation methods can be broadly classified into two categories: **I.** *Visual-Guided*, and **II.** *Direct regression* methods. Visual-Guided methods (Wang et al. 2023; Liang et al. 2024; Xiang et al. 2024) rely on pre-trained 2D diffusion models, which can lead to error accumulation during the subsequent image-to-3D conversion process. While MagicTailor (Zhou et al. 2024) addresses the issue of semantic pollution in the text-to-image stage, the resulting continuous representations still struggle with object-level decoupling. Direct regression methods (Nichol

et al. 2022; Jun and Nichol 2023) focus on generating 3D assets directly from input text. Although 3D object generation has made significant advancements, directly applying a similar philosophy to solve 3D scene generation is still very challenging. Due to the lack of high-quality text-to-3D scene datasets, it hinders the ability to implement a feed-forward network directly. Furthermore, incorporating layout information adds an additional layer of complexity to the scene synthesis process.

LLM driven 3D Scene Generation

As discussed in §Introduction, rule-based methods can utilize explicit layout information to generate accurate and diverse 3D layouts. However, designing an effortless mechanism for layout arrangement remains an open question in 3D scene synthesis. With the reasoning capabilities of LLMs, works such as LayoutGPT (Feng et al. 2023a) and SceneTeller (Öcal et al. 2024) can generate visual layouts and place existing 3D assets for scene synthesis. However, because of the single-step planning strategy, their output often exhibits physical implausibilities, *e.g.,* floating objects or intersecting geometries. Additionally, they tend to perform suboptimally in complex scenes with numerous objects, mainly due to the absence of spatial constraints.

On the other hand, LLM-driven frameworks aim to reduce human effort in the 3D layout arrangement task. To improve layout rationality, recent reinforcement learning-based methods utilize constraint solvers to enhance physical validity. Holodeck (Yang et al. 2024c) and its extensions (Yang et al. 2024b; Sun 2025) translate relational phrases into geometric constraints, which are optimized using gradient-based methods to eliminate collisions. Diffuscene (Tang et al. 2024) and Physcene (Yang et al. 2024a) incorporate physics engines to simulate object dynamics, iteratively adjusting parameters derived from LLMs to ensure stability. Despite advances in realism, their reliance on existing 3D assets limits the versatility and flexibility needed for generating diverse and innovative scenes. Beyond these layout arrangement approaches, our model provides a fully automated pipeline for generating complete and controllable 3D scenes, including individual decoupling objects.

Preliminary

3D Layout Arrangement Taking layout descriptions as input, this problem aims to arrange unordered 3D assets from existing datasets within a 3D environment. Formally, given a layout criterion φ_{layout}, a space defined by four walls oriented along the cardinal directions $\{w_1, \ldots, w_4\}$, and a set of N 3D meshes $\{m_1, \ldots, m_N\}$, the objective is to create a 3D scene that reflects the most accurate spacial relationships of the provided layout configuration. Previous methods (Yang et al. 2024c; Feng et al. 2023b; Sun 2025) treat indoor layout as 3D reasoning tasks, arranging objects in space according to open-ended language instructions. They assume that the input 3D objects are upright and an off-the-shelf vision-language model (VLM), *e.g.,* GPT-4 (OpenAI and other 2024), to determine the front-facing orientations of the objects. As one of the representative methods, LayoutVLM (Sun 2025) annotates each object with a concise textual description s_i, and the dimensions of its axis-aligned bounding box, after rotating to face the $+x$, are represented as $b_i \in \mathbb{R}^3$. The desired output of the layout generation process is the pose of each object $p_i = (x_i, y_i, z_i, \theta_i)$, which includes 3D position and its rotation about the z-axis.

Neural Representation Unlike explicit representations such as meshes or point clouds, implicit neural representation methods train a 3D model through differentiable rendering. There are mainly two types of neural representations: Neural Radiance Fields (Mildenhall et al. 2021) and 3D Gaussian Splatting(3DGS) (Kerbl et al. 2023). 3DGS represents the scene as a set of 3D Gaussians $\{\mathcal{G}_i\}_{i=1}^{M}$, each parametrized with center position $\mu_i \in \mathbb{R}^3$, covariance $\mathbf{\Sigma}_i \in \mathbb{R}^{3\times3}$, color $c_i \in \mathbb{R}^3$, and opacity $\alpha_i \in \mathbb{R}^1$. The 3D Gaussians can be queried by $\mathcal{G}(x) = e^{-\frac{1}{2}(x)^T\Sigma^{-1}(x)}$, where x represents the distance between μ and the query point. The 3D Gaussians are optimized through differentiable rasterization for projection rendering, comparing the resulting image to the training views in the captured dataset using image loss metrics. In the text-to-3D task, there is typically no ground truth (GT) image available. Existing works usually employ a diffusion model to generate pseudo-GT images for distilling the 3D representation.

Diffusion Models Diffusion models (Ho, Jain, and Abbeel 2020; Song, Meng, and Ermon 2020) generate data by iteratively denoising a sample from pure noise. During training, a noisy version of a data sample $\mathbf{x} \sim p_{\text{data}}$ is generated as $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x} + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is standard Gaussian noise, and $\bar{\alpha}_t$ controls noise level. The discrete diffusion timestep t is sampled from a uniform distribution $p_t \sim \mathcal{U}(0, t_{max})$. The denoising network θ predicts the added noise $\boldsymbol{\epsilon}_\theta$ and is optimized with the score matching objective: $\mathcal{L}_t = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}},\, t\sim p_t,\, \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2\right]$.

DreamFusion (Poole et al. 2022) distills 3D representations from a 2D text-to-image diffusion model through Score Distillation Sampling (SDS). Let θ denote the parameters of a differentiable 3D representation, and let g represent a rendering function. The rendered image produced for a given camera pose c can be expressed as $x_0 = g(\theta, c)$, where x_0 is the clean (noise-free) rendering. A noisy view at timestep t is obtained as: $x_t = \alpha_t\, x_0 + \sigma_t\, \boldsymbol{\epsilon}$. Then SDS distills θ through a 2D diffusion model ϕ with frozen parameters as follows:

$$\nabla_\theta \mathcal{L}_{\text{SDS}}(\theta) = \mathbb{E}_{t,\boldsymbol{\epsilon},c}\left[w(t)\,(\boldsymbol{\epsilon}_\phi(\mathbf{x}_t; y, t) - \boldsymbol{\epsilon}) \times \frac{\partial g(\theta, c)}{\partial \theta}\right],$$

where $w(t)$ is a timestep-dependent weight and $\boldsymbol{\epsilon}_\phi$ is the noise estimator of the frozen diffusion model ϕ conditioned on the text prompt y.

Methodology

RoomPlanner is a fully automated pipeline designed to generate complete, complex, and controllable 3D indoor scenes from concise user prompts. As shown in Figure 2, our framework comprises three main components. We begin by introducing a hierarchical LLM-driven planner that reasons and

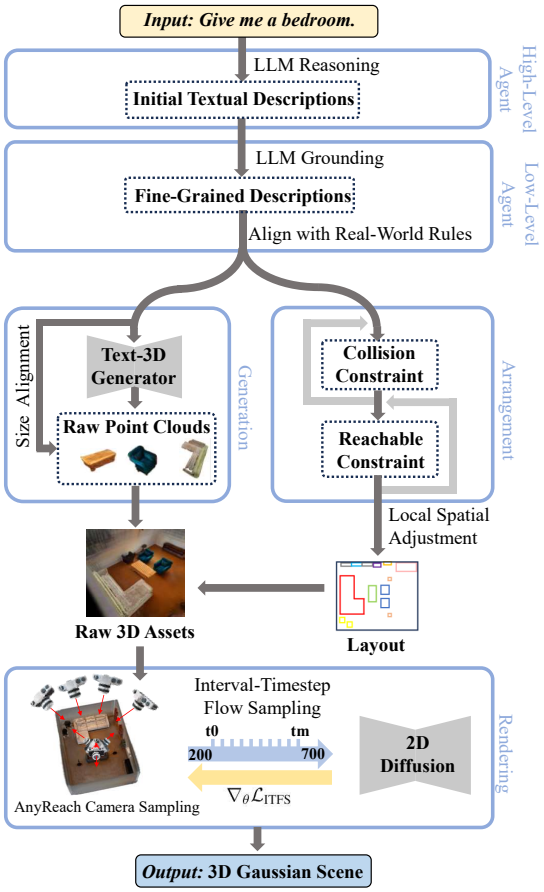


Figure 2: RoomPlanner follows a 'Reasoning-Grounding, Arrangement, and Optimization' pipeline, decomposing the complex 3D scene generation task into scalable subtasks.

grounds the target room type based on the user prompt. It produces detailed textual descriptions $y_i (i = 1, ..., N)$ for each asset and a scene layout configuration y_{scene}. Next, we create the initial 3D point cloud for each asset using a text-to-3D object generator (Jun and Nichol 2023) with y_i as input. Meanwhile, a layout planner computes the optimal sizes and poses for the assets, generating a complete scene map that includes object coordinates, orientations, physical properties, and style descriptions. The assets are then positioned within the coordinate system of the room, according to pose specifications. Finally, we employ an Interval Timestep Flow Sampling with a novel camera sampling strategy to perform single-stage scene optimization, yielding results with realistic visuals and high 3D consistency.

Hierarchical LLM Agents Planning

Given a concise user prompt T, such as *"A bedroom"*, our high-level planner adopts the reasoning capabilities of an LLM agent to semantically expand T and identify relevant object categories for the scene population. Select the appropriate types of objects that align with the scene style specified by the input prompt. The LLM agent also generates

Algorithm 1: RoomPlanner Framework

Input: short-prompt description $\{\mathbf{I}\}$
Output: 3D room scene $\mathbf{Scene}^\star$
// High-Level Reasoning
$(\mathcal{O}, \mathcal{S}) \leftarrow \text{LLMPARSE}(\mathbf{I})$
// Low-Level Grounding
$\mathcal{P} \leftarrow \text{INITPOINTCLOUD}(\mathcal{O}, \mathcal{S})$
$(\hat{\mathcal{O}}, \hat{\mathcal{S}}) \leftarrow \text{ALIGNSIZES}(\mathcal{O}, \mathcal{S}, \mathcal{P})$
// Layout Arrangement I: Collision Constraint
$\varphi_{\text{layout}} \leftarrow \text{ARRANGE}(\hat{\mathcal{O}}, \hat{\mathcal{S}}, \mathcal{P})$
for $t_1 = 1$ to maxTimeIter **do**
  if not $\text{COLLISIONFREE}(\varphi_{\text{layout}})$ **then**
    $\varphi_{\text{layout}} \leftarrow \text{FIXLAYOUT}(\varphi_{\text{layout}}, \textit{collision})$
  else
    break ▷ collision-free layout $\hat{\varphi}_{\text{layout}}$ found
// Layout Arrangement II: Reachability Constraint
for $t_2 = 1$ to maxTimeIter **do**
  if not $\text{REACHABLE}(\hat{\varphi}_{\text{layout}})$ **then**
    $\hat{\varphi}_{\text{layout}} \leftarrow \text{FIXLAYOUT}(\hat{\varphi}_{\text{layout}}, \textit{reachable})$
  else
    break ▷ reachable layout $\varphi^\star_{\text{layout}}$ found
// Differentiable Scene Rendering
$\mathcal{T} \leftarrow \text{ANYREACH}(\varphi^\star_{\text{layout}})$
$\mathbf{Scene}^\star \leftarrow \text{ITFS}(\varphi^\star_{\text{layout}}, \mathcal{T})$

an indexed set (O_i, S_i) for the room. Subsequently, the low-level LLM agent planner is used to enhance these assets with stylistic, textural, and material attributes.

For spatial information, we first adopt Shap-E (Jun and Nichol 2023) to generate the corresponding point cloud representation $\mathbf{p}_i = (x_i, y_i, z_i, c_i)$ for each object O_i according to the index i. We then predict the central position for each object by estimating candidate center coordinates (x_i^c, y_i^c, z_i^c) in the scene. Each object O_i is associated with a semantic label. To ensure semantic and physical plausibility, the physical agent aligns the real-world scale of each object and rescales it if inconsistencies are detected. The updated object parameters and spatial context $(\hat{O}_i, \hat{S}_i)$ are stored in *.json* files for downstream tasks. Further details can be found in the Supplementary Material (Supp).

Layout Arrangement Planning

Taking the detail layout criterion $\varphi_{\text{layout}} = (\hat{O}_i, \hat{S}_i, \mathbf{p}_i)$ as input, we treat the indoor scene generation task as if in a closed space, following the same strategy used in a previous 3D layout arrangement method (Yang et al. 2024c). The process begins with the construction of structural elements such as floors, walls, doors, and windows, followed by the arrangement of floor-mounted furniture and the placement of wall-mounted objects. Next, we employ interactive optimization to update the layout design by verifying whether the generated layout satisfies our two constraints, which are specified as follows.

Collision Constraint Given the inferred spatial constraints φ_{layout} by LLM agents, we first adopt the depth-first-

search algorithm to find a valid placement for object candidates inspired by Holodeck (Yang et al. 2024c). We sequentially place objects according to the inferred symbolic rules, such as positioning objects close to walls, aligning them with other objects(*e.g., nightstands* beside a *bed*), or oriented in specific directions. To enhance layout quality, we further decouple wall-mounted and ground-level object types while ensuring they both follow the same symbolic reasoning process. Specifically, we define a collision-aware layout reward $\mathcal{R}_{\text{coll}}$ to ensure physical feasibility.

$$\mathcal{R}_{\text{coll}} = -(\sum_{i \neq j} \text{IoU}_{3D}(b_i, b_j) + \sum_{i=1}^{N} \sum_{k=1}^{W} \text{IoU}_{3D}(b_i, \text{b}_k^{wall})),$$

where b_i and b_j represent the bounding boxes of objects, and b_k^{wall} denotes the bounding boxes of wall structures. The IoU_{3D} is a metric that measures the discrepancy between two candidate 3D bounding boxes. Note that a layout is valid only when $\mathcal{R}_{\text{coll}} = 0$. In practice, we set a time limit of 30 seconds for the t_1 iterating process; after this period, any objects that are still colliding will be removed.

Reachability Constraint We ensure navigability by requiring all objects to be accessible to a virtual agent represented by the bounding box b^{agent}. Different from the reachability constraints in Physcene (Yang et al. 2024a), we rasterize the 3D Scene Map into a 2D traversable map and employ the A* search algorithm, initiating from a fixed starting point (*e.g.,* a doorway) to each object location. An object is considered reachable only if a valid path exists. The reachability reward $\mathcal{R}_{\text{reach}}$ is defined as follows:

$$\mathcal{R}_{\text{reach}} = -\sum_{i=1}^{W} \text{IoU}_{3D}(b_i, b^{agent}).$$

If an object is deemed unreachable, we undertake local spatial adjustments within a restricted search grid. Again, if no viable solution is identified in 30 seconds, the object will be removed from the layout. This integrated reasoning process ensures both physical plausibility and functional accessibility of the final layout $\varphi^{\star}_{\text{layout}}$.

Differentiable Scene Optimization

AnyReach Camera Trajectory Sampling Proir work AnyHome (Fu et al. 2024) hypothesizes that the camera orients toward the room's center, generating egocentric trajectories that spiral around the scene. Meanwhile, they randomly sample the camera view to capture local details and refine object-level layouts. Unlike AnyHome, our AnyReach sampling strategy supports a more accurate "Zoom-in and Zoom-out" approach to swiftly capture global and local views.

Specifically, we utilize a spiral camera path across the upper hemisphere surrounding each object, with each camera directed toward the object's center. This spiral sampling aims to provide a global observation of the background and objects within the scene. When the "Zoom-in" movement occurs, our camera employs the A* algorithm (Hart, Nilsson, and Raphael 1968) to navigate to the object's location, ensuring the shortest path from the global view to the local object-level view. Taking advantage of A*, our approach conserves camera poses and minimizes the time spent observing irrelevant background such as *wall* along the way. Once the camera reaches the object, we implement a "Zoom-out" movement directly from the current object location to the original global path. This repeated jumping strategy prevents prolonged focus on a single object, ensuring comprehensive view supervision and reducing the object-level geometric blur from distant viewing. More visualization and implementation details are provided in the Supp.

Interval Timestep Flow Sampling In the rendering stage of Figure 2, to enhance the quality of raw 3D scene representations, we propose an Interval-Timestep Flow Sampling (ITFS) strategy that utilizes a range of different timesteps for generating 2D diffusion views. Our approach integrates ITFS with reflection flow matching and multi-timestep sampling, gradually optimizing scene quality from coarse to fine with enhancing semantic details (visualized in Figure 4).

Conditional Flow Matching (CFM) (Lipman et al. 2023) has proven to be a robust framework for training continuous normalizing flows, primarily due to its efficiency in computation and overall effectiveness. To obtain higher-fidelity priors, we replace ϕ with Stable Diffusion 3.5 (Esser et al. 2024a), which utilizes a rectified-flow formulation to model both the forward and reverse processes through a time-dependent vector field. The forward interpolation is defined as $x(t) = (1-t)\,x_0 + t\,\boldsymbol{\epsilon},\ \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and the reverse field $v(x,t) = \frac{\partial x}{\partial t}$ is approximated by a neural network v_ϕ. The training process adheres to the CFM objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,\boldsymbol{\epsilon}}\left[w(t)\,\|v_\phi(x_t;t) - (\boldsymbol{\epsilon} - x_0)\|_2^2\right]$$

Building on the CFM objective function and SDS, we introduce a new **Flow Distillation Sampling (FDS)** strategy. The objective function $\nabla_\theta \mathcal{L}_{\text{FDS}}$ is defined as follows:

$$\mathbb{E}_{t,\boldsymbol{\epsilon}}\Big[w(t)\big(v_\phi(x_t;y,t) - (\boldsymbol{\epsilon} - x_0)\big) \times \frac{\partial g(\theta,c)}{\partial \theta}\Big]$$

Empirically, timesteps $t \in [200, 300]$ emphasize geometric features, while larger timesteps $t > 500$ favor semantic alignment, a pattern also noted by DreamScene (Li et al. 2024a). Since FDS samples in a single timestep, it cannot take advantage of both regimes. To address this limitation, we introduce ITFS which generates intermediate steps from small timestep intervals t_0 to large timestep intervals t_m. This is achieved during the optimization of the objective function $\nabla_\theta \mathcal{L}_{\text{ITFS}}$:

$$\mathbb{E}_{t,\boldsymbol{\epsilon}}\left[\sum_{i=1}^{m} w(t_i)\,\big(v_\phi(x_t;y,t_i) - (\boldsymbol{\epsilon} - x_0)\big) \times \frac{\partial g(\theta,c)}{\partial \theta}\right],$$

where m denotes the value of the sampling numbers. This multi-interval strategy allows early steps to refine geometric features while later steps consolidate semantic information. As a result, it produces 3D scenes that are both structurally accurate and visually coherent.

Type	Method	Layout Planning	End-to-End Generation	Room Scene	Edition	Realistic (object)	Realistic (scene)	Surface Reconstruction
Visual-Guided	Text2Room (Höllein et al. 2023)	-	-	-	-	✓	✓	✓
	SceneWiz3D (Zhang et al. 2024)	-	-	-	✓	-	-	✓
	Pano2Room (Pu, Zhao, and Lian 2024)	-	-	-	✓	-	-	✓
	Director3d (Li et al. 2024b)	-	-	-	✓	-	-	✓
	Scene4U (Huang et al. 2025)	-	-	✓	-	-	-	✓
Rule-based	Set-the-Scene (Cohen-Bar et al. 2023)	✓	-	-	-	✓	✓	✓
	GALA3D (Liu et al. 2023)	✓	✓	-	✓	✓	-	-
	AnyHome (Fu et al. 2024)	✓	-	✓	✓	✓	-	✓
	GraphDreamer (Gao et al. 2024)	✓	✓	-	✓	✓	-	-
	DreamScene (Li et al. 2024a)	✓	✓	-	✓	✓	✓	-
	Ours	✓	✓	✓	✓	✓	✓	✓

Table 1: Comparison of 3D indoor scene synthesis methods, categorized into rule-based and visual-guided approaches. ✓ means capability supported; '-' means without that capability. Our method demonstrates comprehensive capabilities across all metrics.

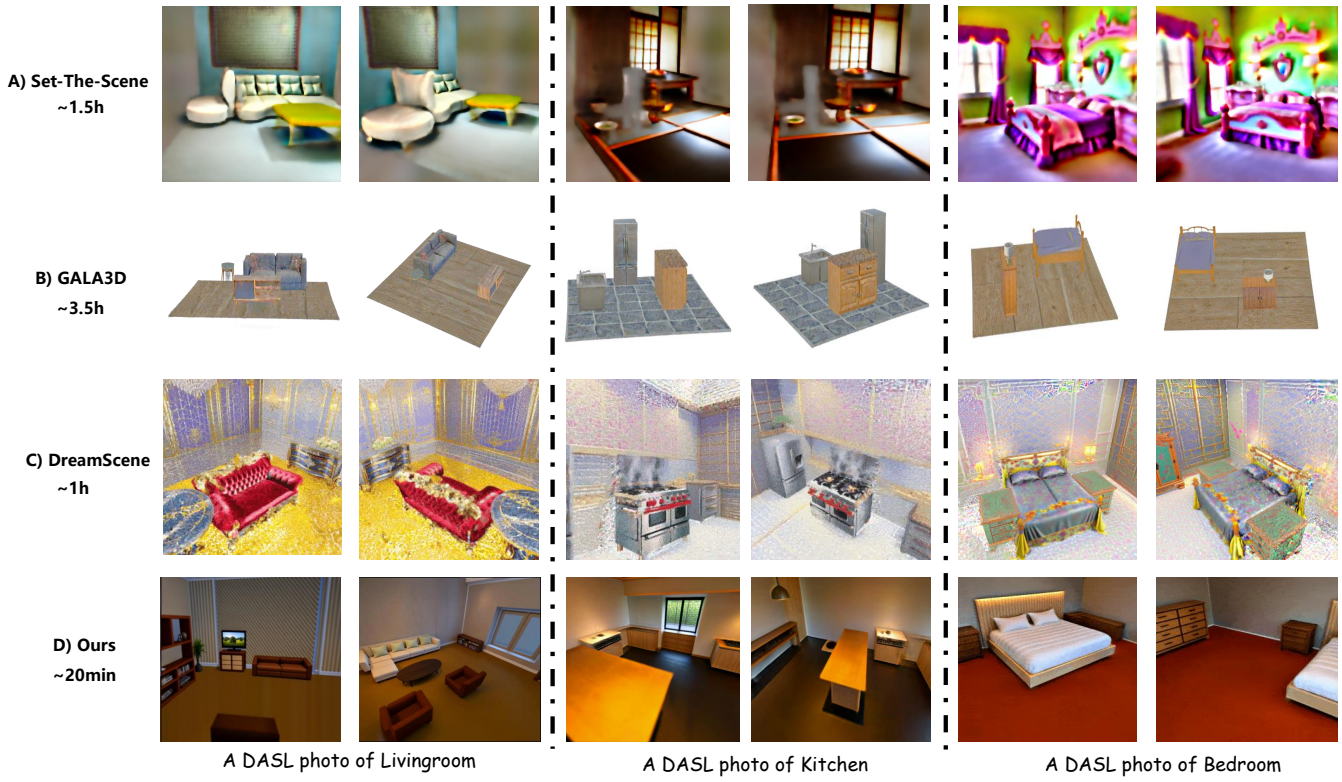


Figure 3: We compare the interactive scene 3D generation with Set-the-Scene (Cohen-Bar et al. 2023), GALA3D (Liu et al. 2023) and DreamScene (Li et al. 2024a).

Experiments

Implementation Details We utilize GPT-4 (OpenAI and other 2024) as our LLM agents for both reasoning and grounding modules. Shap-E (Jun and Nichol 2023) generated point clouds serve as the initial representation of objects, while Stable Diffusion 3.5 Medium (Esser et al. 2024b) provides guidance for rendering 3D scene models. We set the rendering iterations to 1500 and the resolution to 512×512, and ITFS sampling interval value $m = 3$, drawing $t_0 \sim [200, 400]$, $t_1 \sim [400, 600]$ and $t_2 \sim [600, 800]$. We tested RoomPlanner and all baselines on the same NVIDIA 4090 GPU for fair comparison.

Qualitative Results

Figure 3 compares our model with other state-of-the-art methods. We observe that GALA3D (Liu et al. 2023) lacks adequate asset diversity, while Set-the-Scene (Cohen-Bar et al. 2023) shows geometric discontinuities. Additionally,

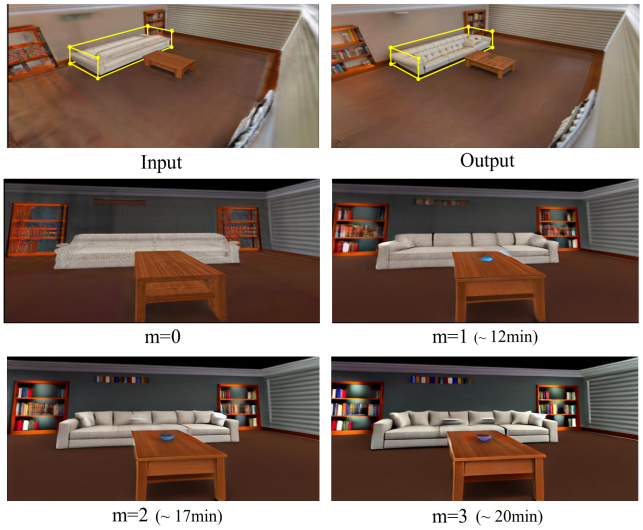


Figure 4: Effectiveness of ITFS. The coach labels in yellow boxes guide the orientation towards a more rational perspective. As the timesteps progress from $m = 0$ to $m = 3$, the scene quality is optimized from a coarse representation to a fine depiction, enriched with more semantic details.

DreamScene (Li et al. 2024a) suffers from particle opacity artifacts and coarse surface geometry, all of which compromise visual fidelity. In contrast, our model generates physically plausible scene synthesis. The decoupled guidance and layout arrangement approaches ensure physical interaction constraints without sacrificing scene diversity. As a result, our model can produce high-quality 3D scenes with object-level decoupling, surpassing prior differentiable approaches in rendering speed, visual quality, and geometric consistency, all while maintaining editability.

Effectiveness of ITFS As illustrated in Figure 4, our method leverages implicit layout knowledge obtained from a 2D generation model to further optimize the 3D scene. The proposed ITFS strategy significantly enhances scene generation by refining object orientation and detail. The coach labels in the two yellow boxes guide the orientation to achieve a more rational and coherent perspective. As the timesteps progress from $m = 0$ to $m = 3$, the scene quality is optimized from a coarse representation to a fine, detailed depiction. This progression not only improves geometric accuracy but also enriches the semantic details, resulting in a more realistic and contextually appropriate rendering.

Quantitative Results

User Study for Semantic Alignment We conducted a user study to compare our approach with existing open source methods, focusing on scene quality and scene authenticity. We selected five common indoor scenes, *ie.,* living room, kitchen, dining room, bedroom, and workplace, for quantitative evaluation. Thirty participants, including professional interior designers, Embodied-AI algorithm engineers, and graduate students, evaluated these scenes on a 1-to-5 scale, indicating their satisfaction from low to high. As shown in Table 2, our method achieved the highest scores across different downstream fields.

Method	User Study		Quality		
	↑ Rationality	↑ Quality	↑ AS	↑ IR	↑ CS %
Set-the-scene	1.13	2.80	4.83	41.10	26.05
GALA3D	2.61	3.22	5.07	42.67	25.40
DreamScene	3.10	3.15	5.13	43.85	26.77
Ours	**4.10**	**3.86**	**5.31**	**48.83**	**28.44**

Table 2: Quantitative evaluation across open soucre method on Indoor Scene Generation. Our method demonstrates consistent superiority across quality metrics (Rationality, Quality) and user study metrics (AS, IR, CS).

3D Scene Generation Quality As shown in Table 2, we tested our approach on three qualitative metrics. Aesthe. Score (AS) (Murray, Marchesotti, and Perronnin 2012) that evaluates aesthetic quality of scene. We also compared 3D generation quality on the ImageReward (T3Bench) (IR) (Xu et al. 2023), which refers to image reward applicantion in T3Bench, and CLIP Score (CS) (Hessel et al. 2021) to evaluation the text-3D scene alignment. Set-the-Scene (Cohen-Bar et al. 2023), GALA3D (Liu et al. 2023) and DreamScene (Li et al. 2024a) both lack of scene diversity and rendering realism issues, bringing negatively increasing their overall scores. Our method performs well on all metrics, which is consistent with the results shown in the user study, which once again proves the superiority of our paradigm combining LLM, scene plan and ITFS generation.

Conclusion

We present RoomPlanner, an automatic framework for text-to-3D scene generation. Following a pipeline of 'Reasoning-Grounding, Arrangement, and Optimization,' our method enhances diversity, complexity, and efficiency for text-to-3D scene generation. By integrating LLMs for scene layout guidance, collision and reachability constraints for layout arrangement, and specific sampling strategies for rendering, our method generates explicit layout criteria to guide rational spatial placements. Experimental results demonstrate that RoomPlanner outperforms existing methods in generating diverse and complex scenes while requiring fewer computational resources for realistic 3D representation.

Limitation Despite its computational efficiency, our method operates on an RTX 4090 GPU with 24GB VRAM. To balance efficiency and performance within this memory constraint, we limit the increase in the number of 3D Gaussians during later optimization stages. This results in insufficient resources to adequately model the fine-grained scattering behavior of light on complex materials and structures. Future work will focus on enhancing visual realism by addressing these limitations, particularly in improving the modeling of complex light transport and recovering finer geometric details, to generate scenes suitable for a broader range of demanding applications.

Appendix Material

Hierarchical LLM Agents Planning

This section provides details about our five prompt templates used for LLM agents to generate long-term descriptions. Each module enforces physical plausibility constraints and syntactic standardization, allowing LLM agents to perform reasoning and grounding.

Floor and Wall Height Module This module structures room layout generation through explicit geometric and material constraints, as shown in Figure 5. Rectangular boundary enforcement, *e.g.,* "each room is a rectangle", eliminates irregular shapes, ensuring compatibility with downstream systems. Coordinate precision, *e.g.,* "units in meters", allows direct conversion from metric to 3D and avoids unit ambiguity. Non-overlap connectivity rule, *e.g.,* "connected, not overlapped", mimics the construction principles of the real world and enforces physically plausible spatial relationships. Size thresholds, *e.g.,* "3–8 m side length, $\leq$ 48 m^2 area", prevent unrealistic room proportions while accommodating standard residential dimensions.

Floor plan Prompt: You are an experienced room designer. Please assist me in crafting a floor plan. Each room is a rectangle. You need to define the four coordinates and specify an appropriate design scheme, including each room's color, material, and texture. Assume the wall thickness is zero. Please ensure that all rooms are connected, not overlapped, and do not contain each other.

- Note: the units for the coordinates are meters.
- For example: living room | maple hardwood, matte | light grey drywall, smooth | [(0, 0), (0, 8), (5, 8), (5, 0)] kitchen | white hex tile, glossy | light grey drywall, smooth | [(5, 0), (5, 5), (8, 5), (8, 0)]
- A room's size range (length or width) is 3m to 8m. The maximum area of a room is 48 m^2. Please provide a floor plan within this range and ensure the room is not too small or too large.
- It is okay to have one room in the floor plan if you think it is reasonable.
- The room name should be unique.

Now, I need a design for {input}. Additional requirements: {additional requirements}. Your response should be direct and without additional text at the beginning or end.

Wall Height Prompt: I am now designing {input}. Please help me decide the wall height in meters. Answer with a number, for example, 3.0. Do not add additional text at the beginning or in the end.

Figure 5: Prompt templates for LLM reasoning to generate the floor and wall height module.

Doorway Prompt: I need assistance in designing the connections between rooms. The connections could be of three types: doorframe (no door installed), doorway (with a door), or open (no wall separating rooms). The sizes available for doorframes and doorways are single (1m wide) and double (2m wide).

Ensure that the door style complements the design of the room. The output format should be: room 1 | room 2 | connection type | size | door style. For example:
exterior | living room | doorway | double | dark brown metal door
living room | kitchen | open | N/A | N/A
living room | bedroom | doorway | single | wooden door with white frames

The design under consideration is {input}, which includes these rooms: {rooms}.
The length, width, and height of each room in meters are: {room_sizes}
Certain pairs of rooms share a wall: {room_pairs}. There must be a door to the exterior.
Adhere to these additional requirements {additional_requirements}.

Provide your response succinctly, without additional text at the beginning or end.

Figure 6: Prompt templates for LLM reasoning to generate the doorway module.

Doorway Module This module regulates inter-room connectivity through parametric constraints and contextual style alignment, as shown in Figure 6. Typology limitation, *e.g.,* "doorframe/doorway/open", eliminates architecturally invalid connections, which can reduce the output of hallucinations. Dimensional catalog, *e.g.,* "1m/2m enforces the compliance of the building code. Style binding rule, *e.g.,* "complements room design", ensures material coherence. It can prevent contradictions, such as "industrial metal door" connecting to "traditional Japanese tatami room".

Window Module This module optimizes window design through constrained parametric selection and context-aware placement rules, as shown in Figure 7. The type-size catalog (fixed/hung/slider with standardized dimensions) ensures the manufacture of specifications, eliminating invalid configurations in the LLM outputs. Unified style constraints, *e.g.,* "within the same room, all windows must be

Window Prompt: Guide me in designing the windows for each room. The window types are: fixed, hung, and slider. The available sizes (width x height in cm) are: fixed: (92, 120), (150, 92), (150, 120), (150, 180), (240, 120), (240, 180) hung: (87, 160), (96, 91), (120, 160), (130, 67), (130, 87), (130, 130) slider: (91, 92), (120, 61), (120, 91), (120, 120), (150, 92), (150, 120)
Your task is to determine the appropriate type, size, and quantity of windows for each room, bearing in mind the room's design, dimensions, and function.
Please format your suggestions as follows: room | wall direction | window type | size | quantity | window base height (cm from floor). For example: living room | west | fixed | (130, 130) | 1 | 50
I am now designing input. The wall height is wall height cm. The walls available for window installation (direction, width in cm) in each room are: walls Please note: It is not mandatory to install windows on every available wall. Within the same room, all windows must be the same type and size. Also, adhere to these additional requirements: additional requirements.
Provide a concise response, omitting any additional text at the beginning or end.

Figure 7: Prompt templates for LLM reasoning to generate the window module.

the same type and size", maintain architectural coherence while reducing combinatorial complexity. Base height parameter, *e.g.,* "50cm–120cm" aligns with human anthropometry standards, preventing implausible placements such as floor-level windows in bathrooms.

Object Selection Module This prompt structures scene furnishing through constraint systems that enforce physical plausibility and realism, as shown in Figure 8. Spatial hierarchy constraint, *e.g.,* "small objects on top" optimizes space utilization while avoiding visual monotony or clutter. Isolation constraint, *e.g.,* "Do not provide rug/mat, windows, doors, curtains, and ceiling objects, *etc.*", eliminates cross-module conflicts and prevents duplicate assets. Dual-stage output structuring constraint, *e.g.,* "first use natural language to explain high-level design strategy, then follow *.json* file format", ensures schema consistency.

Object Alignment Module This module enables the generation of vivid architectural scenes and eliminates common LLM hallucination patterns, as shown in Figure 9. Material anchoring requirement, *e.g.,* "clearly mention floor and wall materials", eliminates ambiguous descriptions such as "nice floors" in the LLM output. Structural purity constraint, *e.g.,* "no people/objects except architectural features", restricts the semantic space and allows focus on consistency of the wall, windows, and posters.

Object Grounding Prompt: You are an experienced room designer, please assist me in selecting large **floor/wall** objects and small objects on top of them to furnish the room. You need to select appropriate objects to satisfy the customer's requirements. You must provide a description and desired size for each object since I will use it to retrieve object. If multiple identical items are to be placed in the room, please indicate the quantity and variance type (same or varied). Present your recommendations in JSON format: "sofa": "description": "modern sectional, light grey sofa", "location": "floor", "size": [100, 80, 200], "quantity": 1, "variance type": "same", "objects on top": ["object name": "news paper", "quantity": 2, "variance type": "varied", "object name": "pillow", "quantity": 2, "variance type": "varied", "object name": "mobile phone", "quantity": 1, "variance type": "same"] , "tv stand": "description": "a modern style TV stand", "location": "floor", "size": [200, 50, 50], "quantity": 1, "variance type": "same", "objects on top": ["object name": "49 inch TV", "quantity": 1, "variance type": "same", "object name": "speaker", "quantity": 2, "variance type": "same", "object name": "remote control for TV", "quantity": 1, "variance type": "same"] , "painting": "description": "abstract painting", "location": "wall", "size": [100, 5, 100], "quantity": 2, "variance type": "varied", "objects on top": [] , "wall shelf": "description": "a modern style wall shelf", "location": "wall", "size": [30, 50, 100], "quantity": 1, "variance type": "same", "objects on top": ["object name": "small plant", "quantity": 2, "variance type": "varied", "object name": "coffee mug", "quantity": 2, "variance type": "varied", "object name": "book", "quantity": 5, "variance type": "varied"]
Currently, the design in progress is **INPUT**, and we are working on the **ROOM TYPE** with the size of ROOM SIZE. Please also consider the following additional requirements: REQUIREMENTS.
Here are some guidelines for you:
1. Provide a reasonable type/style/quantity of objects for each room based on the room size to make the room not too crowded or empty. 2. Do not provide rug/mat, windows, doors, curtains, and ceiling objects that have been installed for each room.
3. I want more types of large objects and more types of small objects on top of the large objects to make the room look more vivid. Please first use natural language to explain your high-level design strategy for **ROOM TYPE**, and then follow the desired JSON format **strictly** (do not add any additional text at the beginning or end).

Figure 8: Prompt templates for LLM grounding in object selection.

Object Grounding Prompt: Generate a concise prompt (under 35 words) describing a description featuring floor and wall, creating a vivid and realistic atmosphere. Focus on architectural details. The prompt must:

- Clearly mention the floor and wall materials.
- Convey a vivid, realistic atmosphere.
- Contain no people or objects other than architectural features (like windows, walls, posters).
- Return only the plain prompt text | no quotation marks, markdown, headers, or explanations.

Figure 9: Prompt templates for LLM grounding in object alignment.

AnyReach Camera Trajectory Sampling

In this section, we present a detailed demonstration of how AnyReach camera trajectory sampling is implemented during the rendering stage.

Rendering Parameters In Listing 1, we provide parameter values that are directly employed in our method, as referenced throughout the experimental implementation.

Zoom-in Mode Trajectory Figure 10 demonstrates our "zoom-in" camera sample strategy generated using grid-based spatial reasoning. The sampling mode first discretizes the room into an occupancy grid, where cells are marked as obstructed (*e.g.,* by furniture or wall) or navigable. Starting from the doorway, it plans sequential paths to key object centers using the A* algorithm, ensuring human collision-free movement through the room. The camera is positioned at the height of the human eye with a randomized elevation, while its orientation remains fixed to a central room target. The resulting trajectory simulates natural human movement in the real world, prioritizing the acquisition of local object-level details.

Zoom-out Mode Trajectory Figure 11 shows the "Zoom-out" spherical camera trajectory mode. It utilizes a spiral camera path, providing global observation of the background and objects within the scene through angular variations in azimuth and elevation. The system dynamically selects observation targets as the closest object center to each camera position. Each camera is directed toward the object's center through geometric alignment calculations, creating systematic inspection orbits. This approach generates comprehensive coverage of both background context and object layouts while maintaining collision-aware positioning within valid room boundaries.

Hybrid Mode Camera Trajectory Figure 12 shows our hybrid sampling alternates between spiral global observation (zoom-out) and A* optimized object approach (zoom-in), creating a dynamic "observe-approach-reset" cycle. This pattern minimizes idle transit time compared to sequential sampling while maintaining collision awareness through continuous occupancy grid validation during all transitions.

Listing 1: Rendering Parameters

```yaml
# total training steps
iters: 1500
densification_interval: 100
density_start_iter: 100
density_end_iter: 1500
# Batch size
batch_size: 1
# Learning rates
position_lr_init: 0.0008
position_lr_final: 2.5e-05
position_lr_delay_mult: 0.01
position_lr_max_steps: 1200
feature_lr: 0.01
feature_lr_final: 0.005
opacity_lr: 0.01
opacity_reset_interval: 30000
scaling_lr: 0.005
scaling_lr_final: 0.005
rotation_lr: 0.001
rotation_lr_final: 0.0004
texture_lr: 0.3
geom_lr: 0.0002
# Point cloud growth / sparsification
max_point_number: 3000000
percent_dense: 0.01
densify_grad_threshold: 0.001
density_thresh: 1.0
# Camera and viewing parameters
# vertical FOV range (deg)
fovy: [76.0,96.0]
elevation: 0
# camera orbit radius range
radius:[1.5,2.5]
# yaw range (deg)
ver: [-50,50]
# File paths and resources
mesh_format: glb
outdir: null
plyload: null
# Prompt settings
prompt: ""
# Scene statistics
room_size: ""
room_num:
assets_num:
# Object size list with prompts
object_size_list:
  - object: 1
    object_size:
    prompt: ""
# Miscellaneous flags
save: true
sh_degree: 3
```

Editing Functionality

RoomPlanner supports the suitable placement of new objects (*e.g.,* "a table with three cups") into existing 3D scenes. It automatically resolves spatial conflicts while maintaining

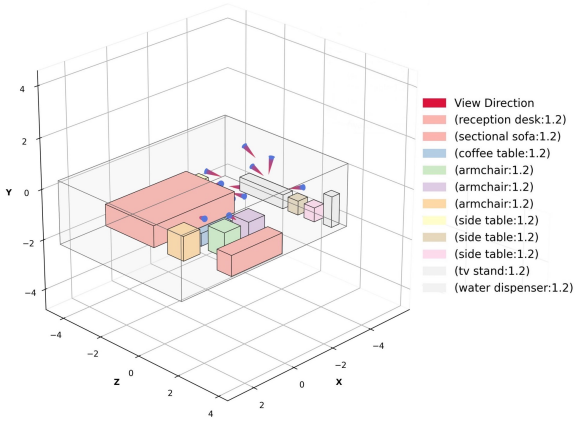


Figure 10: Camera trajectory sampling of **zoom-in** mode.

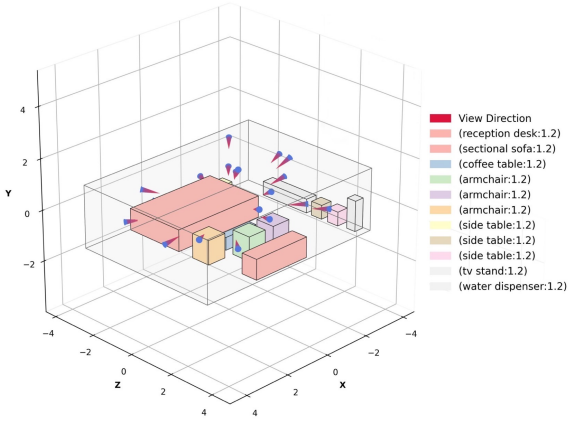


Figure 11: Camera trajectory sampling of **zoom-out** mode.

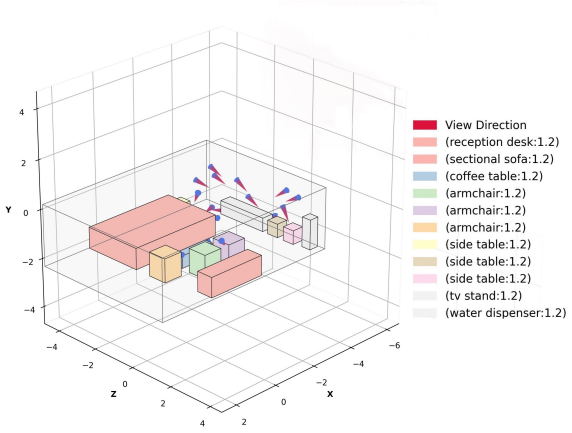


Figure 12: Camera trajectory sampling of **hybrid** mode.

realistic proportions and physical coherence. As shown in Figure 13, the added cups are adapted to the scale of the

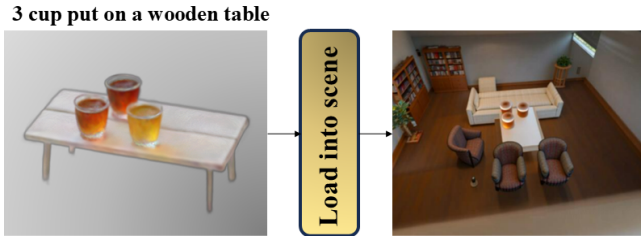


Figure 13: Visualization of editing functionality for additional items.

scene and adhere to the rules of placement in the real world without manual intervention.

Diversity of Generated Scenes

In Figure 14, we present 25 additional scenes generated by RoomPlanner. These examples encompass a wide range of room types, including common spaces such as kitchens, dining rooms, living rooms, and bedrooms, as well as less typical environments like robot labs, music labs, prison cells, and art rooms. This diverse selection highlights capability of RoomPlanner to generate realistic and varied 3D interiors across both everyday and uncommon scenarios.

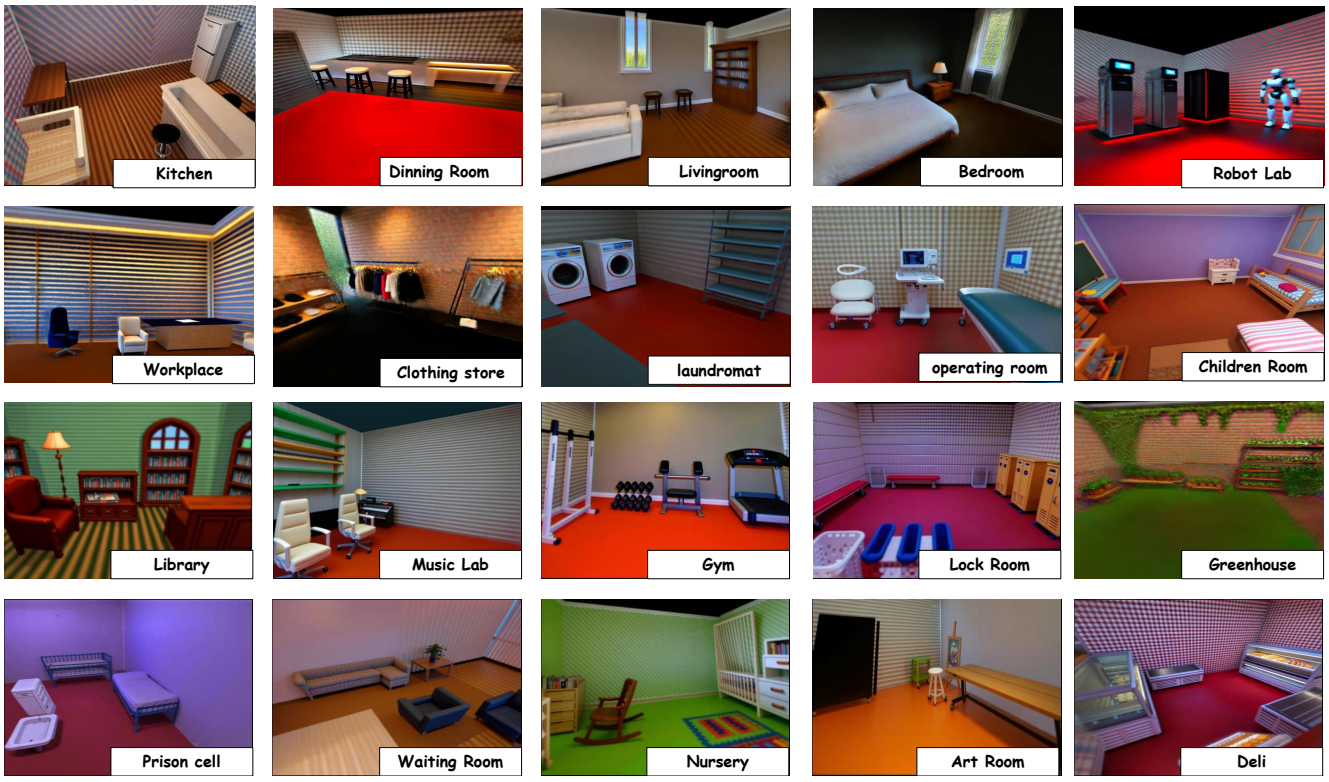


Figure 14: A diverse set of 3D scenes generated by RoomPlanner, demonstrating its ability to create both common and uncommon room types.

References

2025. LayoutVLM: Differentiable Optimization of 3D Layout via Vision-Language Models. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 29469–29478.

Chen, Z.; Walsman, A.; Memmel, M.; Mo, K.; Fang, A.; Vemuri, K.; Wu, A.; Fox, D.; and Gupta, A. 2024. URDFormer: A Pipeline for Constructing Articulated Simulation Environments from Real-World Images. *arXiv preprint arXiv:2405.11656*.

Cohen-Bar, D.; Richardson, E.; Metzer, G.; Giryes, R.; and Cohen-Or, D. 2023. Set-the-Scene: Global-Local Training for Generating Controllable NeRF Scenes.

Esser, P.; Kulal, S.; Blattmann, A.; Entezari, R.; Müller, J.; Saini, H.; Levi, Y.; Lorenz, D.; Sauer, A.; Boesel, F.; et al. 2024a. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*.

Esser, P.; Kulal, S.; Blattmann, A.; Entezari, R.; Müller, J.; Saini, H.; Levi, Y.; et al. 2024b. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. In *Forty-First International Conference on Machine Learning (ICML)*.

Feng, W.; Zhu, W.; Fu, T.-j.; Jampani, V.; Akula, A.; He, X.; Basu, S.; Wang, X. E.; and Wang, W. Y. 2023a. Layoutgpt: Compositional visual planning and generation with large language models. *Advances in Neural Information Processing Systems*, 36: 18225–18250.

Feng, W.; Zhu, W.; jui Fu, T.; Jampani, V.; Akula, A.; He, X.; Basu, S.; Wang, X. E.; and Wang, W. Y. 2023b. LayoutGPT: Compositional Visual Planning and Generation with Large Language Models. *arXiv preprint arXiv:2305.15393*.

Fu, R.; Wen, Z.; Liu, Z.; and Sridhar, S. 2024. Anyhome: Open-vocabulary generation of structured and textured 3d homes. In *European Conference on Computer Vision*, 52–70. Springer.

Gao, G.; Liu, W.; Chen, A.; Geiger, A.; and Schölkopf, B. 2024. Graphdreamer: Compositional 3d scene synthesis from scene graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 21295–21304.

Hart, P. E.; Nilsson, N. J.; and Raphael, B. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2): 100–107.

Hessel, J.; Holtzman, A.; Forbes, M.; Bras, R. L.; and Choi, Y. 2021. CLIPScore: A Reference-free Evaluation Metric for Image Captioning. In *EMNLP*.

Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 6840–6851.

Höllein, L.; Cao, A.; Owens, A.; Johnson, J.; and Nießner, M. 2023. Text2Room: Extracting Textured 3D Meshes from 2D Text-to-Image Models. 7909–7920.

Huang, Z.; He, J.; Ye, J.; Jiang, L.; Li, W.; Chen, Y.; and Han, T. 2025. Scene4U: Hierarchical Layered 3D Scene Reconstruction from Single Panoramic Image for Your Immerse Exploration. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 26723–26733.

Jun, H.; and Nichol, A. 2023. Shap-E:Generating Conditional 3D Implicit Functions. arXiv:2305.02463.

Kerbl, B.; Kopanas, G.; Leimkühler, T.; and Drettakis, G. 2023. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4): 1–14.

Li, H.; Shi, H.; Zhang, W.; Wu, W.; Liao, Y.; Wang, L.; Lee, L.-h.; and Zhou, P. Y. 2024a. Dreamscene: 3d gaussian-based text-to-3d scene generation via formation pattern sampling. In *European Conference on Computer Vision*, 214–230. Springer.

Li, X.; Lai, Z.; Xu, L.; Qu, Y.; Cao, L.; Zhang, S.; Dai, B.; and Ji, R. 2024b. Director3D: Real-world Camera Trajectory and 3D Scene Generation from Text. *arXiv:2406.17601*.

Liang, Y.; Yang, X.; Lin, J.; Li, H.; Xu, X.; and Chen, Y. 2024. Luciddreamer: Towards high-fidelity text-to-3d generation via interval score matching. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 6517–6526.

Lipman, Y.; Chen, R. T. Q.; Ben-Hamu, H.; Nickel, M.; and Le, M. 2023. Flow Matching for Generative Modeling. In *The Eleventh International Conference on Learning Representations*.

Liu, R.; Wu, R.; Hoorick, B. V.; Tokmakov, P.; Zakharov, S.; and Vondrick, C. 2023. GALA3D: Towards text-to-3D complex scene generation via layout-guided diffusion. *arXiv preprint*, arXiv:2303.16969.

Maillard, L.; Sereyjol-Garros, N.; Durand, T.; and Ovsjanikov, M. 2024. DeBaRA: Denoising-Based 3D Room Arrangement Generation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Mildenhall, B.; Srinivasan, P. P.; Tancik, M.; Barron, J. T.; Ramamoorthi, R.; and Ng, R. 2021. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1): 99–106.

Murray, N.; Marchesotti, L.; and Perronnin, F. 2012. AVA: A large-scale database for aesthetic visual analysis. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2408–2415.

Nichol, A.; Jun, H.; Dhariwal, P.; Mishkin, P.; and Chen, M. 2022. Point-e: A system for generating 3d point clouds from complex prompts. *arXiv preprint arXiv:2212.08751*.

Nießner, M.; Zollhöfer, M.; Izadi, S.; and Stamminger, M. 2013. Real-time 3D reconstruction at scale using voxel hashing. *ACM Transactions on Graphics*, 32(6): 1–11.

Öcal, B. M.; Tatarchenko, M.; Karaoğlu, S.; and Gevers, T. 2024. SceneTeller: Language-to-3D Scene Generation. In *European Conference on Computer Vision*, 362–378. Springer.

OpenAI; and other. 2024. GPT-4 Technical Report. arXiv:2303.08774.

Poole, B.; Jain, A.; Barron, J. T.; and Mildenhall, B. 2022. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv*.

Pu, G.; Zhao, Y.; and Lian, Z. 2024. Pano2room: Novel view synthesis from a single indoor panorama. In *SIGGRAPH Asia 2024 Conference Papers*, 1–11.

Song, J.; Meng, C.; and Ermon, S. 2020. Denoising Diffusion Implicit Models. In *arXiv preprint*, volume arXiv:2010.02502. Published in ICLR 2021.

Sun, Q.; Zhou, H.; Zhou, W.; Li, L.; and Li, H. 2024. Forest2seq: Revitalizing order prior for sequential indoor scene synthesis. In *European Conference on Computer Vision*, 251–268. Springer.

Tang, J.; Nie, Y.; Markhasin, L.; Dai, A.; Thies, J.; and Nießner, M. 2024. Diffuscene: Denoising diffusion models for generative indoor scene synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 20507–20518.

Wang, T.; Mao, X.; Zhu, C.; Xu, R.; Lyu, R.; Li, P.; Chen, X.; Zhang, W.; Chen, K.; Xue, T.; et al. 2024. Embodiedscan: A holistic multi-modal 3d perception suite towards embodied ai. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 19757–19767.

Wang, Z.; Lu, C.; Wang, Y.; Bao, F.; Li, C.; Su, H.; and Zhu, J. 2023. ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation. *arXiv*.

Xiang, J.; Lv, Z.; Xu, S.; Deng, Y.; Wang, R.; Zhang, B.; Chen, D.; Tong, X.; and Yang, J. 2024. Structured 3D Latents for Scalable and Versatile 3D Generation. *arXiv preprint arXiv:2412.01506*.

Xu, J.; Liu, X.; Wu, Y.; Tong, Y.; Li, Q.; Ding, M.; Tang, J.; and Dong, Y. 2023. ImageReward: learning and evaluating human preferences for text-to-image generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 15903–15935.

Yang, Y.; Jia, B.; Zhi, P.; and Huang, S. 2024a. Physcene: Physically interactable 3d scene synthesis for embodied ai. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16262–16272.

Yang, Y.; Lu, J.; Zhao, Z.; Luo, Z.; Yu, J. J.; Sanchez, V.; and Zheng, F. 2024b. Llplace: The 3d indoor scene layout generation and editing via large language model. *arXiv preprint arXiv:2406.03866*.

Yang, Y.; Sun, F.-Y.; Weihs, L.; VanderBilt, E.; Herrasti, A.; Han, W.; Wu, J.; et al. 2024c. Holodeck: Language Guided Generation of 3D Embodied AI Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16227–16237.

Yu, H.-X.; Duan, H.; Herrmann, C.; Freeman, W. T.; and Wu, J. 2025. WonderWorld: Interactive 3D Scene Generation from a Single Image. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 5916–5926.

Zhang, Q.; Wang, C.; Siarohin, A.; Zhuang, P.; Xu, Y.; Yang, C.; Lin, D.; Zhou, B.; Tulyakov, S.; and Lee, H.-Y. 2024. Towards Text-guided 3D Scene Composition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 6829–6838.

Zhou, D.; Huang, J.; Bai, J.; Wang, J.; Chen, H.; Chen, G.; Hu, X.; and Heng, P.-A. 2024. MagicTailor: Component-Controllable Personalization in Text-to-Image Diffusion Models. *arXiv preprint arXiv:2410.13370*.